

Betriebliche Projektarbeit

Fachinformatiker Anwendungsentwicklung

Abschlussprüfung Winter 2025/26

Entwicklung einer webbasierten Anwendung zur Buchung von
Büro-Arbeitsplätzen im Shared-Desk-Modell

Prüflingsbewerber

Milena Suvajac
Spessartstr. 1
36119 Neuhof

Prüflingsnummer

20122

Ausbildungsbetrieb

EDAG Engineering GmbH
Reesbergstr. 1
36039 Fulda

24. November 2025

Persönliche Erklärung

Ich versichere durch meine Unterschrift, dass ich das zugrundeliegende Projekt und diesen Bericht zur betrieblichen Projektarbeit (Projektdokumentation) selbstständig und ohne fremde Hilfe angefertigt habe.

Alle fremden Quellen wie Softwaremodule, Veröffentlichungen, andere Projektberichte und Zusatzarbeiten, die ich funktionsmäßig, wörtlich oder auch nur sinngemäß übernommen habe, sind von mir als solche gekennzeichnet worden.

Die Veröffentlichung oder Vervielfältigung dieser Projektdokumentation an Dritte ist ohne ausdrückliche Erlaubnis des Unternehmens EDAG Engineering GmbH nicht gestattet.

Die Arbeit hat in dieser Form keiner anderen Prüfungsinstitution vorgelegen.

Ort, Datum

Unterschrift

Das Projekt wurde

☐ abgenommen

☐ nicht abgenommen

Ort, Datum

Projektverantwortlicher

Stempel des Betriebes

Inhaltsverzeichnis

Abbildungsverzeichnis

Tabellenverzeichnis

1 Einleitung

Einleitung

Diese Projektdokumentation wurde im Rahmen der Industrie- und Handelskammer (IHK)-Abschlussprüfung zum Fachinformatiker für Anwendungsentwicklung erstellt. Der Ausbildungsbetrieb ist die EDAG Engineering GmbH, ein internationaler Engineering-Dienstleister für die Automobilindustrie. Der Autor arbeitet in der Abteilung Mobility Informationstechnik (IT), die IT-Lösungen und Softwareplattformen für moderne Mobilitätskonzepte entwickelt, mit Schwerpunkt auf Digitalisierung und Vernetzung.

Ziel des Projekts ist die Entwicklung einer webbasierten Anwendung zur Buchung von Büroarbeitsplätzen im Shared-Desk-Modell, um die transparente Planung und Reservierung von Arbeitsplätzen zu ermöglichen. Das System basiert auf einem Full-Stack-Ansatz mit Spring Boot Backend, PostgreSQL-Datenbank und Next.js Frontend, containerisiert in Docker Container Plattform (Docker).

Fachbegriffe und Abkürzungen werden im Abkürzungsverzeichnis erläutert.

1.1 Projektumfeld

Ich absolviere derzeit eine Ausbildung zum Fachinformatiker für Anwendungsentwicklung, die ich im Rahmen meiner betrieblichen Tätigkeit bei der EDAG Engineering GmbH durchführe. Die EDAG Engineering GmbH ist ein international tätiger Engineering-Dienstleister im Bereich Mobilität und Teil der EDAG Group, die weltweit mit mehreren tausend Mitarbeitenden vertreten ist. Das Unternehmen bietet Entwicklungs- und IT-Dienstleistungen für führende Automobilhersteller und Industriekunden an und deckt dabei sämtliche Phasen des Produktentstehungsprozesses ab.

Das Projekt wird in der Abteilung Mobility IT umgesetzt, die sich auf die Entwicklung und Integration digitaler Lösungen für interne und externe Kundenprojekte spezialisiert hat. Diese Abteilung verantwortet unter anderem die Konzeption, Entwicklung und Betreuung von Softwarelösungen, welche die Geschäftsprozesse innerhalb der EDAG Group effizient unterstützen.

Im Zuge des zunehmenden hybriden Arbeitens hat die EDAG Engineering GmbH ein Shared-Desk-Modell eingeführt, bei dem mehr Mitarbeitende als physische Büroarbeitsplätze vorhanden sind. Aktuell existiert jedoch kein zentrales System zur transparenten Planung und Reservierung von Arbeitsplätzen. Die Organisation erfolgt bisher informell, was regelmäßig zu Unklarheiten über verfügbare Arbeitsplätze und Anwesenheiten führt.

Zur Verbesserung dieser Situation wird im Rahmen dieser Projektarbeit eine webbasierte Anwendung zur internen Buchung von Büro-Arbeitsplätzen entwickelt. Diese soll eine intuitive Übersicht über verfügbare Arbeitsplätze bieten und Mitarbeitenden ermöglichen, Plätze im Voraus zu reservieren.

Die technische Umsetzung erfolgt als Full-Stack-Webanwendung mit einem Spring-Boot-Backend, einer PostgreSQL-Datenbank und einem Next.js-Frontend. Das System wird containerisiert in ei-

ner Docker-Umgebung realisiert. Die Anwendung ist für den internen Gebrauch innerhalb der EDAG Engineering GmbH konzipiert und wird zunächst in der Abteilung Mobility IT eingesetzt. Eine spätere Ausweitung auf weitere Unternehmensbereiche ist technisch vorgesehen, jedoch nicht Bestandteil dieses Projekts.

1.2 Projektziel

Ziel des Projekts ist die Entwicklung einer webbasierten Anwendung zur internen Buchung von Büro-Arbeitsplätzen innerhalb der EDAG Engineering GmbH. Die Anwendung soll Mitarbeitenden ermöglichen, verfügbare Arbeitsplätze tagesgenau einzusehen, zu reservieren und Buchungen transparent zu verwalten. Dadurch wird die Auslastung der vorhandenen Büroflächen optimiert und die Planung im Rahmen des Shared-Desk-Modells vereinfacht.

Im Mittelpunkt steht die Umsetzung eines Minimum Viable Product (MVP), das die Kernfunktionen eines Arbeitsplatzbuchungssystems bereitstellt:

- Anzeige verfügbarer Arbeitsplätze über eine Kalenderansicht,
- Reservierung und Verwaltung von Buchungen,
- Benutzerfreundliche Oberfläche zur intuitiven Bedienung,
- Zentrale Speicherung der Buchungsdaten in einer PostgreSQL-Datenbank.

Das System soll zunächst lokal betrieben werden und dient als funktionaler Prototyp für den späteren produktiven Einsatz innerhalb der EDAG Engineering GmbH.

1.3 Projektbegründung

Die aktuelle Verwaltung der Büroarbeitsplätze erfolgt in der Abteilung Mobility IT der EDAG Group ohne ein zentrales System. Mitarbeitende koordinieren ihre Anwesenheitstage informell, was regelmäßig zu Unklarheiten über die Verfügbarkeit und Belegung von Arbeitsplätzen führt. Besonders an Tagen mit hoher Büroanwesenheit entstehen dadurch organisatorische Probleme und eine ineffiziente Nutzung der vorhandenen Büroflächen.

Die Entwicklung einer webbasierten Buchungsanwendung soll diesen Zustand verbessern, indem sie Transparenz und Planbarkeit schafft. Durch die digitale Reservierung von Arbeitsplätzen können Doppelbelegungen vermieden und freie Plätze effizient genutzt werden. Gleichzeitig wird der Kommunikationsaufwand für die Mitarbeitenden reduziert, da alle relevanten Informationen zentral abrufbar sind.

Darüber hinaus leistet die Lösung einen Beitrag zur besseren Umsetzung des Shared-Desk-Konzepts im Rahmen des hybriden Arbeitens. Führungskräfte erhalten durch die Anwendung eine übersichtliche Datengrundlage zur Analyse der Arbeitsplatznutzung, was eine optimierte Flächenplanung ermöglicht. Somit trägt das Projekt sowohl zur organisatorischen als auch zur wirtschaftlichen Effizienz des Unternehmens bei.

1.4 Projektschnittstellen

In diesem Kapitel werden die Schnittstellen der Anwendung beschrieben. Diese lassen sich in personelle, organisatorische und technische Schnittstellen unterteilen.

1.4.1 Personelle Schnittstellen

Während der Projektdurchführung steht Herr Henning als technischer Ansprechpartner und Projektbetreuer zur Verfügung. Er unterstützt bei fachlichen Rückfragen sowie bei der Abstimmung mit der IT-Abteilung. Die spätere Nutzung der Anwendung erfolgt durch die Mitarbeitenden der Abteilung Mobility IT am Standort Petersberg (Fulda). Sie fungieren zugleich als Pilotanwender und geben Rückmeldung zur Bedienbarkeit und Funktionsweise der Lösung.

1.4.2 Organisatorische Schnittstellen

Die Anwendung wird in das bestehende organisatorische Umfeld der EDAG Group eingebettet. Eine Abstimmung mit der internen IT-Abteilung ist erforderlich, um sicherzustellen, dass die Anwendung innerhalb des Unternehmensnetzwerks betrieben werden kann und die Sicherheitsrichtlinien eingehalten werden.

Darüber hinaus wird die Personalabteilung in die Planung der Nutzerrollen einbezogen, um sicherzustellen, dass nur autorisierte Mitarbeitende Zugriff auf bestimmte Funktionen (z. B. Admin-Bereich) erhalten.

1.4.3 Wirtschaftliche Schnittstellen

Da es sich um ein internes Entwicklungsprojekt handelt, entstehen keine externen Lizenz- oder Beschaffungskosten. Die benötigten Ressourcen (Server, Entwicklungsumgebung, Containerplattform) werden von der bestehenden EDAG-IT-Infrastruktur bereitgestellt. Eventuelle Entscheidungen über eine spätere Ausweitung auf weitere Abteilungen werden in Abstimmung mit dem Management getroffen.

1.4.4 Technische Schnittstellen

Die Anwendung ist ein eigenständiges System, das innerhalb des internen Firmennetzwerks betrieben wird. Sie interagiert mit folgenden technischen Systemen: Externe Systeme oder Schnittstellen zu bestehenden Anwendungen bestehen im Rahmen dieses Projekts nicht.

1.5 Projektabgrenzung

Der Projektumfang ist auf 80 Stunden begrenzt. Um eine erfolgreiche Umsetzung innerhalb dieses Rahmens sicherzustellen, wurden folgende Abgrenzungen festgelegt:

- Single Sign-On (SSO): Eine spätere Integration in das zentrale Authentifizierungssystem der EDAG Group ist grundsätzlich vorgesehen, gehört jedoch nicht zum Leistungsumfang des Projekts. Für das MVP erfolgt die Anmeldung lokal über einfache Benutzerverwaltung.

-
- Mobile Nutzung: Die Anwendung wird für die Desktop-Nutzung im Browser entwickelt. Eine mobile Version oder App ist nicht Bestandteil dieses Projekts.
 - Produktiver Betrieb: Die Bereitstellung auf einem zentralen Server oder in der produktiven Infrastruktur der EDAG Group ist nicht Teil des Projekts. Die Anwendung wird lokal in einer Docker-Umgebung betrieben.
 - Erweiterte Funktionen: Funktionen wie die Integration in Outlook-Kalender, Benachrichtigungssysteme oder die automatische Auswertung von Nutzungsstatistiken sind konzeptionell vorgesehen, werden jedoch nicht implementiert.

Diese Abgrenzungen gewährleisten, dass der Fokus des Projekts auf der Entwicklung einer funktionsfähigen, lokal lauffähigen Basisversion der Anwendung liegt, die als Grundlage für eine spätere Weiterentwicklung und Integration in das Unternehmensumfeld dient.

2 Analysephase

2.1 Projektphasen mit Zeitplanung

Nr.	Vorgangsname	Dauer	Anfang	Fertigstellen	Vorgänger
	Anforderungsspezifikation	8 Std.	–	–	–
1	Rahmenbedingungen	2 Std.	Mo 03.11.25	Mo 03.11.25	–
2	Nutzeranforderungen	3 Std.	Mo 03.11.25	Mo 03.11.25	1
3	Zieldefinition	1 Std.	Mo 03.11.25	Mo 03.11.25	2
4	Ist-Analyse	2 Std.	Mo 03.11.25	Mo 03.11.25	3
	Projektplanung	12 Std.	–	–	–
5	Konzept / Architektur	3 Std.	Di 04.11.25	Di 04.11.25	4
6	Technologieauswahl	2 Std.	Di 04.11.25	Di 04.11.25	5
7	DB-Modellierung	1 Std.	Di 04.11.25	Do 06.11.25	6
8	UI-Mockups	3 Std.	Do 06.11.25	Do 06.11.25	7
9	Schnittstellen / Rechte	1 Std.	Do 06.11.25	Do 06.11.25	8
10	Zeit- und Aufgabenplan	1 Std.	Do 06.11.25	Do 06.11.25	9
11	Kostenabschätzung	1 Std.	Do 06.11.25	Do 06.11.25	10
	Projektdurchführung	44 Std.	–	–	–
12	Entwicklungsumgebung	4 Std.	Fr 07.11.25	Fr 07.11.25	11
13	DB-Implementierung	4 Std.	Fr 07.11.25	Fr 07.11.25	12
14	REST-API	10 Std.	Mo 10.11.25	Mo 10.11.25	13
15	Frontend	14 Std.	Di 11.11.25	Do 13.11.25	14
16	Integration FE/BE	6 Std.	Do 13.11.25	Do 13.11.25	15
17	Tests (Unit / Int.)	6 Std.	Mo 17.11.25	Mo 17.11.25	16
	Projektabschluss und Kontrolle	16 Std.	–	–	–
18	Funktionstests	2 Std.	Mo 17.11.25	Di 18.11.25	17
19	QS / Code-Review	2 Std.	Di 18.11.25	Di 18.11.25	18
20	Techn. Doku	5 Std.	Do 20.11.25	Do 20.11.25	19
21	Benutzerhandbuch	2 Std.	Fr 21.11.25	Fr 21.11.25	20
22	Bewertung / Fazit	1 Std.	Mo 27.11.25	Mo 27.11.25	21
23	Präsentationsvorb.	4 Std.	Do 27.11.25	Fr 27.11.25	22
Gesamtzeit					80 Std.

Tabelle 2.1: Zeitplanung des Projekts „Webbasierte Arbeitsplatzbuchung“

2.2 Ressourcenplanung

Folgende Ressourcen werden für die Umsetzung des Projekts benötigt oder sind bereits vorhanden:

Hardware

- 1 × Firmenlaptop des Auszubildenden mit Entwicklungsumgebung
- 1 × EDAG-Server (bestehende Infrastruktur, vorgesehen für den späteren produktiven Betrieb – nicht Teil dieses Projekts)

Software

- IntelliJ IDEA Community Edition – Entwicklungsumgebung für das Backend
- Visual Studio Code – Entwicklungsumgebung für das Frontend
- Docker CLI – Containerisierung und Verwaltung von Frontend, Backend und Datenbank
- PostgreSQL – relationale Datenbank zur Verwaltung der Buchungsdaten
- Next.js – Framework für das Webfrontend
- React Konva – Rendering und Interaktion zur grafischen Darstellung von Räumen
- shadcn/ui – Komponentenbibliothek zur Umsetzung der Benutzeroberfläche
- next-intl – Internationalisierung des Webfrontends (Deutsch/Englisch)
- Spring Boot – Framework für das Backend (REST-API)
- Keycloak – Authentifizierungs- und Autorisierungsserver
- Swagger (OpenAPI) – API-Dokumentation
- Git – Versionsverwaltung
- Figma – Erstellung von UI-Mockups
- Markdown / LaTeX – Projektdokumentation

Personal

- 1 × Auszubildender (Fachinformatiker Anwendungsentwicklung) – Entwicklung und Dokumentation
- 1 × Mitarbeitende der Abteilung Mobility IT – Bereitstellung von Anforderungen und Test-feedback

2.3 Kostenplanung und Kalkulation

Ressource	Berechnung	Kosten (€)
Personalkosten Auszubildender	80 h × 35,00 €	2.800,00
Personalkosten Kollege (Test)	3 h × 75,00 €	225,00
Gesamtkosten		3.025,00 €

Tabelle 2.2: Kostenaufstellung inklusive Stromverbrauchsberechnung

Für Softwarelizenzen oder Hardware entstehen keine weiteren Ausgaben, da alle benötigten Ressourcen bereits durch die EDAG Group bereitgestellt werden.

2.4 Make-or-Buy-Entscheidung

Im Rahmen der Projektplanung wurde geprüft, ob für die Buchung von Büroarbeitsplätzen eine bestehende Standardlösung eingesetzt oder eine individuelle Software entwickelt werden soll. Diese Entscheidung wurde auf Basis technischer, wirtschaftlicher und organisatorischer Kriterien getroffen.

Vergleichskriterien

- **Kosten:** Kommerzielle Arbeitsplatzbuchungssysteme bewegen sich typischerweise zwischen 1,50 € und 3,70 € pro Nutzer und Monat. Beispiele: desk.ly (ab 1,65 €), PULT (ab 1,90 €), deskbird (2,50–3,50 €), Flexopus (1,59–2,99 €) und Yoffix (ab 1,50 € je nach Modul). Viele Anbieter stellen zudem individuelle Enterprise-Pakete bereit, deren Preise abhängig von Funktionsumfang, Ressourcen und Nutzerzahl separat kalkuliert werden. Für rund 80 Mitarbeitende ergeben sich jährliche Lizenzkosten von etwa 1.500 € bis 3.500 €. Die Eigenentwicklung verursacht einmalige Entwicklungskosten von ca. 3.025 € und keine laufenden Lizenzgebühren.
- **Externe Speicherung:** Cloudbasierte Systeme speichern Daten häufig außerhalb der eigenen Unternehmensinfrastruktur. Der Anbieter übernimmt dadurch Teile von Betrieb, Backup und Zugriffssicherung.
- **Eingeschränkte Kontrolle:** Bei externen Lösungen besteht nur begrenzter Einfluss auf Updates, Sicherheitsmaßnahmen und Backup-Prozesse.
- **Abhängigkeiten:** Fremdsysteme führen zu Abhängigkeiten von Preisstruktur, Funktionsumfang und Verfügbarkeit des Anbieters. Eine Eigenentwicklung vermeidet dies vollständig und ermöglicht interne Steuerung von Weiterentwicklungen.
- **Anpassbarkeit:** Standardlösungen bieten nur eingeschränkte Anpassungsmöglichkeiten. Eine individuell entwickelte Lösung kann nahtlos in bestehende EDAG-Systeme integriert und flexibel um zukünftige Funktionen erweitert werden (z.B. Outlook-Integration oder Anwesenheitsanalysen).

Entscheidung

Nach Abwägung aller Faktoren wurde die Entscheidung für die **Make-Variante** getroffen. Die Eigenentwicklung bietet langfristig geringere Kosten, volle Kontrolle über Daten und Funktionen sowie eine bessere Integration in die interne IT-Landschaft der EDAG Group. Die Implementierung im Rahmen dieses Projekts bildet die Grundlage für eine spätere unternehmensweite Nutzung.

2.5 Amortisationsrechnung

Für die Wirtschaftlichkeitsbetrachtung wird die statische Amortisationsrechnung („Durchschnittsmethode“) verwendet. Diese Methode geht davon aus, dass der jährliche Rückfluss konstant bleibt.

Die Amortisationsdauer ergibt sich aus dem Verhältnis von Investitionskosten zu jährlichem Rückfluss:

$$\text{Amortisationsdauer} = \frac{\text{Investitionskosten}}{\text{jährlicher Rückfluss}}$$

2.5.1 Ausgangsdaten

Wie in den Vergleichskriterien dargestellt, bewegen sich die jährlichen Lizenzkosten kommerzieller Buchungssysteme zwischen 1.500 € und 3.500 € bei rund 80 Nutzern. Der Durchschnitt dieser beiden Werte beträgt 2.500 € pro Jahr. Für die Amortisationsrechnung wird dieser Durchschnittswert als jährliche Einsparung herangezogen, da er die realistischste Vergleichsbasis darstellt.

- Investitionskosten Eigenentwicklung: 3,025 €
- Jährliche Lizenzkosten (Minimum): 1,500 €
- Jährliche Lizenzkosten (Maximum): 3,500 €
- Jährliche Lizenzkosten (Durchschnitt): 2,500 €
- Jährlicher Rückfluss (eingesparte Lizenzkosten): 2,500 €

Damit ergibt sich:

$$\text{Amortisationsdauer} = \frac{3,025}{2,500} = 1,21 \text{ Jahre}$$

Die Eigenentwicklung amortisiert sich somit nach rund 1,2 Jahren (entspricht etwa 14–15 Monaten). Ab diesem Zeitpunkt entstehen keine weiteren Kosten, während bei kommerziellen Lösungen kontinuierlich Lizenzgebühren anfallen.

2.5.2 Grafische Darstellung der Amortisation

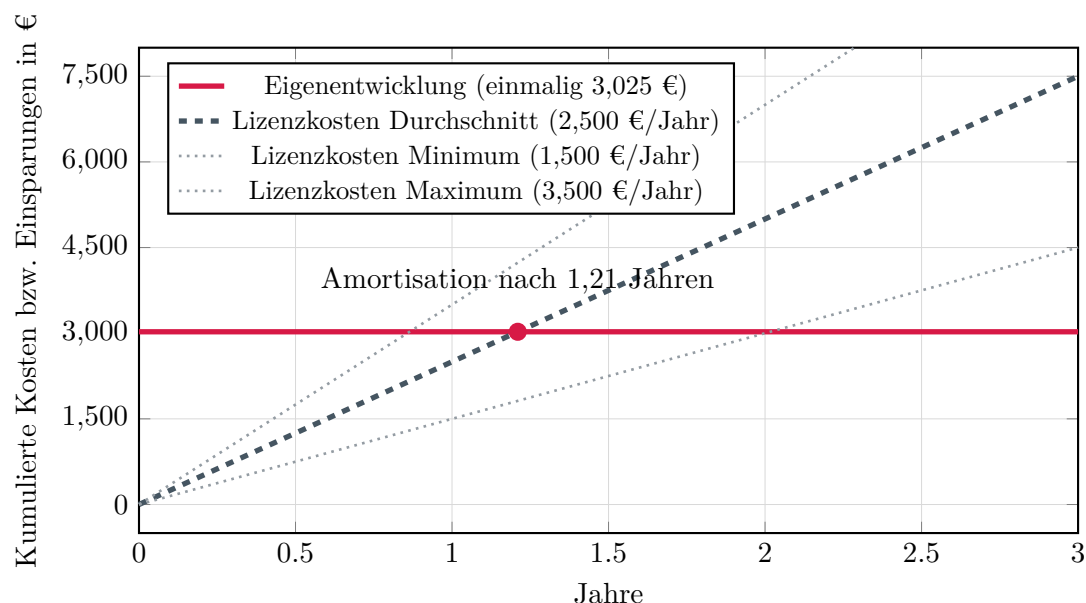


Abbildung 2.1: Amortisationsvergleich: Eigenentwicklung vs. kommerzielle Lizenzkosten

Die Abbildung zeigt, dass die einmaligen Investitionskosten der Eigenentwicklung (rote horizontale Linie bei 3,025 €) nach 1,21 Jahren von den kumulierten Lizenzkosten erreicht werden. Ab diesem Zeitpunkt übersteigen die laufenden Kosten einer kommerziellen Lösung die Investition in die Eigenentwicklung. Bei durchschnittlichen Lizenzkosten von 2,500 € pro Jahr entstehen nach drei Jahren Gesamtkosten von 7,500 €, während die Eigenentwicklung weiterhin bei einmalig 3,025 € bleibt.

2.6 Ist-Analyse

Im Zuge des hybriden Arbeitens hat die EDAG Engineering GmbH am Standort Petersberg (Fulda) ein Shared-Desk-Modell eingeführt, bei dem die Anzahl der Büroarbeitsplätze geringer ist als die Zahl der Mitarbeitenden. Eine feste Zuordnung von Arbeitsplätzen existiert nicht, sodass Mitarbeitende bei Bedarf einen Platz im Büro nutzen können.

Aktuell existiert kein zentrales System zur Planung oder Reservierung von Arbeitsplätzen. Die Zuteilung erfolgt informell und ohne transparente Übersicht. Mitarbeitende entscheiden erst vor Ort, welchen Arbeitsplatz sie an dem jeweiligen Tag nutzen, abhängig davon, welche Plätze gerade frei sind.

Diese spontane Vorgehensweise führt insbesondere an Tagen mit hoher Büroanwesenheit zu Unklarheiten und ineffizienter Nutzung der verfügbaren Büroflächen. Mitarbeitende wissen häufig nicht, welche Arbeitsplätze belegt sind oder welche Kolleginnen und Kollegen anwesend sein werden. Dadurch wird die individuelle Planung erschwert, und es entstehen unnötige Such- und Abstimmungszeiten.

Zusätzlich fehlen historische und statistische Daten zur Auslastung der Arbeitsplätze, wodurch eine datenbasierte Optimierung der Raumbelegung nicht möglich ist.

Die Analyse des aktuellen Zustands zeigt somit, dass der bestehende Prozess nicht effizient, unstrukturiert und fehleranfällig ist. Eine digitale Lösung zur zentralen Buchung und Verwaltung von Arbeitsplätzen ist erforderlich, um die Transparenz zu erhöhen, den organisatorischen Aufwand zu reduzieren und die vorhandenen Büroflächen optimal zu nutzen.

3 Projektplanung

3.1 Vorgehensmodell

Für die Umsetzung des Projekts wird das **Wasserfallmodell** verwendet. Dieses Vorgehensmodell teilt das Projekt in aufeinanderfolgende Phasen auf, die jeweils abgeschlossen werden, bevor die nächste beginnt. Dadurch wird eine strukturierte und nachvollziehbare Entwicklung ermöglicht.

Das Wasserfallmodell wurde gewählt, weil die Anforderungen zu Beginn des Projekts klar definiert sind und eine phasenweise, zeitlich planbare Umsetzung innerhalb des vorgegebenen Rahmens von 80 Stunden ermöglicht wird.

3.1.1 Entwurf der Benutzeroberfläche

Um benutzerfreundliche Oberflächen zu erstellen, wurden Mockups entworfen. Für die Erstellung wurde die Software Figma verwendet. Zunächst wurden notwendige Benutzeroberflächen anhand der Anwendungsfälle ermittelt.

Beim Entwurf der Benutzeroberfläche wurden Farben aus dem EDAG-Stil (Dunkelgrau, Rot, Hellgrau) übernommen.

Die Mockups zeigen die Hauptansichten der Anwendung:

- Raum-Übersicht mit grafischer 2D-Darstellung der Arbeitsplätze
- Buchungsdialog mit Datums- und Zeitauswahl
- Buchungsverwaltung mit tabellarischer Auflistung
- Anwesenheitsverwaltung mit Status-Auswahl
- Admin-Bereich für Raum- und Arbeitsplatzverwaltung

Die grafische Darstellung der Räume erfolgt mithilfe von React Konva, einer Canvas-basierten Rendering-Bibliothek. Arbeitsplätze werden einheitlich in Hellgrau dargestellt, der Status wird durch den Rahmen visualisiert.

3.1.2 Architekturdesign

Die Gesamtarchitektur der Anwendung besteht aus vier Hauptkomponenten, die über definierte Schnittstellen miteinander kommunizieren:

- **Frontend (Next.js):** Präsentationsschicht mit React-Komponenten für die Benutzeroberfläche
- **Backend (Spring Boot):** Representational State Transfer (REST)-Application Programming Interface (API) mit Geschäftslogik in Hexagonaler Architektur

- **Datenbank (PostgreSQL):** Persistenzschicht für alle Anwendungsdaten
- **Keycloak:** Identity Provider für Authentifizierung und Autorisierung über Open Authorization 2.0 (OAuth2)/OpenID Connect (OIDC)

Alle Komponenten werden als Docker-Container betrieben und über ein gemeinsames Docker-Netzwerk verbunden. Die Kommunikation erfolgt über Hypertext Transfer Protocol (HTTP)/REST (Frontend ↔ Backend), Java Database Connectivity (JDBC) (Backend ↔ Datenbank) und OAuth2/OIDC (Frontend/Backend ↔ Keycloak).

Bei der Auswahl des Backend-Architekturdesigns wurde auf Austauschbarkeit und Testbarkeit geachtet, weshalb sich das Backend an der Hexagonalen Architektur (Ports & Adapters) orientiert. Diese Architektur ermöglicht eine vollständige Entkopplung der Domänenschicht von den Infrastruktur-Schichten. Dadurch wird die fachliche Logik in der Domäne gekapselt und die Persistenzschicht sowie externe Schnittstellen austauschbar gehalten. Im Falle einer Umstellung auf eine andere Datenbank oder zusätzliche Client-Anwendungen könnte auf die bestehende Domäne aufgebaut werden, ohne diese anzupassen.

Frontend-Architektur

Die Präsentationsschicht wird mit Next.js und React umgesetzt. Das Frontend ist in folgende Komponenten unterteilt:

- **User Interface (UI)-Komponenten:** Wiederverwendbare React-Komponenten für Buchungen, Räume, Authentifizierung und Layout
- **Custom Hooks:** Wiederverwendbare Logik für API-Aufrufe und State-Management
- **API Proxy:** Zentrale Schnittstelle zur Backend-Kommunikation

Backend-Architektur (Hexagonale Architektur)

Das Backend folgt dem Prinzip der Hexagonalen Architektur mit drei Schichten:

- **Domain Layer:** Geschäftslogik, Entitäten als Plain Old Java Objects (POJOs) (User, Room, Workspace, Booking, DeskType, UserRoom, UserPresence), Domain Services (BookingService, WorkspaceService, RoomService, UserRoomService, UserPresenceService, UserService) sowie abstrakte Repository-Ports.
- **Application Layer:** Implementierung der Use Cases (Inbound Ports), Commands für Schreiboperationen, Queries für komplexe Lesevorgänge sowie Data Transfer Objects (DTOs) zur Entkopplung der REST-API vom Domain-Model.
- **Infrastructure Layer:** REST-Controller (Mapping zwischen DTOs, Commands/Queries und Use Cases), Java Persistence API (JPA)-Repository-Adapter als Implementierung der Repository-Ports, Entity-Mapping zwischen Domain Models und JPA-Entities, Security-Konfiguration (OAuth2/JSON Web Token (JWT)), Cross-Origin Resource Sharing (CORS)-Konfiguration.

Dependency Injection ermöglicht vollständige Inversion of Control. JPA/Hibernate übernimmt das Object-Relational Mapping (ORM).

Komponentendiagramm siehe Anhang (??).

3.1.3 Anwendungsfälle

Zur Darstellung der fachlichen Anwendungsfälle wurden mithilfe der Unified Modeling Language (UML) drei Anwendungsfalldiagramme erstellt – jeweils eines für jede Rolle (siehe Anhang ??). Diese Aufteilung nach Benutzerrollen verbessert die Übersichtlichkeit und ermöglicht eine klare Abgrenzung der Zuständigkeiten.

Das System unterscheidet drei Akteure mit hierarchischer Rollenvererbung:

- **Benutzer (7 Use Cases):** Grundlegende Funktionen zur Arbeitsplatzbuchung, Anwesenheitsverwaltung und Team-Übersicht
- **Raumadministrator (8 Use Cases):** Erbt Benutzer-Rechte und kann zusätzlich Arbeitsplätze in zugewiesenen Räumen verwalten
- **Systemadministrator (5 Use Cases):** Erbt alle Rechte und verwaltet zusätzlich Räume und globale Systemeinstellungen

Da jeder Anwendungsfall eine Authentifizierung über Keycloak (OAuth2/OIDC) voraussetzt, wurde diese technische Abhängigkeit aus Gründen der Übersichtlichkeit nicht explizit in den Diagrammen dargestellt. Detaillierte Use-Case-Diagramme mit vollständigen Beschreibungen befinden sich im Anhang (siehe Abschnitt ??).

3.1.4 Datenmodell

Basierend auf der Anforderungsanalyse und den Mockups wurden sieben Entitäten abgeleitet: **User** (Benutzerkonten), **Room** (Räume), **Workspace** (Arbeitsplätze), **Booking** (Reservierungen), **DeskType** (Arbeitsplatztypen), **UserRoom** (n:m-Relation für Raumadministratoren) und **UserPresence** (Anwesenheitsstatus). Das relationale Datenbankmodell befindet sich in Third Normal Form (3NF) und nutzt JPA/Hibernate für das ORM.

Die Kernentitäten sind mit Primary Keys (PKs) versehen und über Foreign Keys (FKs) verbunden. Workspace enthält zweidimensional (2D)-Koordinaten für die grafische Raumdarstellung. Wichtige Beziehungen: User 1:n Booking, Room 1:n Workspace, User n:m Room (über UserRoom).

Das vollständige Entity-Relationship (ER)-Diagramm mit allen Attributen, Datentypen, Constraints und Kardinalitäten befindet sich im Anhang (siehe Abschnitt ??).

3.1.5 Geschäftslogik

Zur Veranschaulichung der Geschäftslogik wurde das Aktivitätsdiagramm Arbeitsplatz buchen erstellt (siehe Anhang ??). Der Buchungsprozess umfasst Authentifizierung, Verfügbarkeitsprüfung und Konflikterkennung.

Die Geschäftslogik ist in Service-Klassen (BookingService, WorkspaceService, RoomService) gekapselt. Services kommunizieren über Repository-Interfaces mit der Datenbankschicht und geben

DTOs anstelle von Entitäten zurück, um die fachliche Logik zu schützen und die API-Schnittstelle zu entkoppeln.

Zentrale Logikkomponenten:

- **Verfügbarkeitsberechnung:** Dynamische Statusermittlung basierend auf Buchungen, Zuweisungen und Anwesenheitsstatus
- **Konfliktprüfung:** Mehrschichtige Validierung (Zeitraum-Überschneidungen, User-Doppelbuchungen, Zuweisungsbeschränkungen)
- **Fehlerbehandlung:** Sprechende Fehlermeldungen mit HTTP-Statuscodes (400 Bad Request, 404 Not Found, 409 Conflict)
- **Rollenbasierte Logik:** Unterschiedliche Berechtigungen für Benutzer, Raumadministratoren und Systemadministratoren

3.1.6 Maßnahmen zur Qualitätssicherung

Zur Sicherstellung der Softwarequalität wurde eine mehrstufige Teststrategie implementiert:

Unit-Tests: 41 Tests mit JUnit 5 und Mockito für Backend-Services (Code-Coverage: 72%).

Fokus auf kritische Geschäftsregeln wie Buchungskonflikte und Validierungslogik.

API-Tests: Systematische Tests aller REST-Endpunkte mit Swagger UI. Überprüfung von Erfolgs- und Fehlerszenarien sowie HTTP-Statuscodes.

Frontend-Tests: Manuelle Funktionstests für Browser-Kompatibilität und React-Konva-Raumvisualisierung.

Code-Qualität: Statische Analyse durch Integrated Development Environments (IDEs), ESLint für TypeScript, einheitliche Coding-Standards.

Versionsverwaltung: Git mit Feature-Banches und strukturierten Commit-Konventionen.

3.1.7 Lokale Entwicklungsumgebung

Für die lokale Entwicklung und das Testen wurde Docker Compose eingesetzt. Diese Container-basierte Umgebung ermöglicht eine konsistente und reproduzierbare Entwicklungsumgebung auf dem Entwicklungsrechner. Docker Compose orchestriert vier Container (Frontend, Backend, PostgreSQL, Keycloak) mit definierter Startup-Reihenfolge via Healthchecks.

Da es sich um ein MVP handelt, wurde auf ein produktives Deployment-Setup verzichtet. Die Anwendung läuft ausschließlich lokal und dient als funktionaler Prototyp. Ein späterer produktiver Betrieb auf zentralen Servern ist technisch vorgesehen, aber nicht Teil dieses Projekts.

4 Projektdurchführung

Anhand der erstellten Entwürfe aus Kapitel 3 (Entwurfsphase) konnte mit der Implementierung der einzelnen Elemente begonnen werden. Die Implementierung erfolgte gemäß der in der Planung definierten Phasen und Zeitvorgaben.

4.1 Aufsetzen des Projekts

Zu Beginn wurden die Projektstrukturen für Backend und Frontend erstellt. Für das Backend wurde mit Spring Initializr ein Spring Boot 3.2.1-Projekt mit Maven als Build-Tool erzeugt. Anschließend wurden alle notwendigen Abhängigkeiten in der `pom.xml`-Datei hinzugefügt, darunter Spring Data JPA, Spring Security, Spring Web, PostgreSQL-Treiber und SpringDoc OpenAPI für die API-Dokumentation.

Für das Frontend wurde mit Node Package Manager (npm) `create-next-app@latest` ein Next.js 15-Projekt initialisiert. Alle benötigten Abhängigkeiten wurden in der `package.json` eingetragen, einschließlich React Konva für die grafische Raumdarstellung, shadcn/ui für UI-Komponenten, next-intl für Internationalisierung und Tailwind Cascading Style Sheets (CSS) für das Styling.

Um die Anwendung in Docker lauffähig zu machen, wurden Dockerfiles für Backend und Frontend erstellt. Mithilfe der Dockerfiles können Docker-Images gebaut werden, die dann in Docker-Containern ausgeführt werden können.

4.1.1 Docker-Compose-Konfiguration

Da die Webanwendung aus vier Services besteht (Frontend, Backend, PostgreSQL, Keycloak), wurde eine `docker-compose.yml`-Datei erstellt. Docker Compose vereinfacht das Betreiben einer Multi-Container-Anwendung erheblich.

Bei Ausführung mit dem Befehl `docker-compose up -d` werden alle Container gestartet. Durch die `depends_on`-Bedingungen wird sichergestellt, dass die Container in der richtigen Reihenfolge gestartet werden:

1. **PostgreSQL-Container:** Datenbank auf Port 5432 mit Healthcheck
2. **Keycloak-Container:** Identity Provider auf Port 8081, wartet auf PostgreSQL
3. **Backend-Container:** Spring Boot API auf Port 8080, wartet auf PostgreSQL und Keycloak
4. **Frontend-Container:** Next.js auf Port 3000, wartet auf Backend

Für die Entwicklung werden keine persistenten Daten außerhalb der Container benötigt, jedoch wurden Docker-Volumes für `db-data`, `keycloak-data` und `frontend-node-modules` definiert, um die Daten über Container-Neustarts hinweg zu erhalten.

Damit der Zugriff zwischen den Containern funktioniert, wurden alle Container in einem gemeinsamen Docker-Netzwerk `shared_desk_net` platziert. Dadurch können die Container untereinander über ihre Service-Namen kommunizieren (z.B. kann das Backend mit `jdbc:postgresql://db:5432/sharedesk` auf die Datenbank zugreifen).

4.1.2 Keycloak-Konfiguration und Realm-Import

Keycloak wird als Docker-Container betrieben und beim Start automatisch konfiguriert. Das Keycloak-Dockerfile basiert auf dem offiziellen `quay.io/keycloak/keycloak:24.0`-Image und integriert einen Custom Event Listener Provider.

Dieser Provider wird als Maven-Projekt innerhalb des Docker-Build-Prozesses kompiliert und in das Verzeichnis `/opt/keycloak/providers/` kopiert. Der Custom Provider ermöglicht die automatische Synchronisation von Keycloak-Benutzern mit der Anwendungsdatenbank durch Event-basierte Webhooks.

Beim Start des Keycloak-Containers wird durch den Befehl `start-dev -import-realm` automatisch die Realm-Konfiguration aus der Datei `realm-shared-desk.json` importiert. Diese Konfiguration enthält:

- Realm-Name: `shared-desk`
- Rollen: `USER`, `ADMIN`
- Zwei OAuth2-Clients:
 - `shared-desk-frontend` – Public Client für Frontend (OAuth2 PKCE Flow)
 - `shared-desk-admin` – Service Account für Backend (User-Synchronisation mit Keycloak)
- Event-Listener-Konfiguration für automatische User-Synchronisation

4.2 Implementierung der Domäne

Nachdem das Projekt aufgesetzt wurde, konnten aus dem zuvor erstellten ER-Diagramm (siehe Anhang ??) alle benötigten Klassen und Attribute abgeleitet werden. Jeder Entitätstyp im ERM repräsentiert eine Klasse im Domain Layer der Hexagonalen Architektur.

4.2.1 Domain-Modelle als POJOs

Die Domain-Modelle wurden zunächst als einfache POJOs (Plain Old Java Objects) im Package `org.shared_desk.domain.model` implementiert. Diese Klassen sind frei von technischen Abhängigkeiten und enthalten ausschließlich fachliche Logik und Attribute.

Folgende Entitäten wurden erstellt: `User`, `Room`, `Workspace`, `Booking`, `DeskType`, `UserRoom` und `UserPresence`. Jede Entität erhielt eine eindeutige ID als Primärschlüssel. Zur Reduzierung von Boilerplate-Code wurde die Bibliothek Lombok eingesetzt, wodurch Getter, Setter, Konstruktoren und `toString()`-Methoden automatisch generiert werden.

4.2.2 JPA-Entities für Persistierung

Damit die Domain-Modelle auf ein relationales Datenbankmodell abgebildet werden können, wurde jede Klasse mit der JPA-Annotation `@Entity` versehen. Durch diese Annotation kann JPA eine Klasse als Entitätstyp erkennen und mithilfe des ORMs (Object-Relational Mapping) auf eine relationale Datenbank abbilden.

Die Primärschlüssel wurden mit `@Id` und `@GeneratedValue(strategy = GenerationType.IDENTITY)` annotiert, wodurch die Datenbank automatisch fortlaufende IDs vergibt.

4.2.3 Automatische Schema-Generierung

Das Datenbankschema wird automatisch durch Spring Boot und Hibernate generiert. In der `application.yml` wurde die Konfiguration `spring.jpa.hibernate.ddl-auto: update` hinterlegt. Diese Einstellung sorgt dafür, dass beim Start des Backends alle benötigten Tabellen automatisch angelegt oder aktualisiert werden.

Beim ersten Start des Backends wurden folgende sieben Tabellen erstellt: `users`, `rooms`, `desk_types`, `workspaces`, `bookings`, `user_room` und `user_presence`. Alle Tabellen wurden mit den korrekten Primärschlüsseln, Fremdschlüsseln, Constraints und Indizes angelegt.

4.3 Implementierung der Geschäftslogik

Nachdem alle Entitäten abgeleitet wurden, konnte die Domäne um die Services erweitert werden. Die Services bilden die Anwendungsfälle ab (siehe Anhang ??).

4.3.1 Repository-Interfaces

Um den Methoden der Services den Zugriff auf die Datenbank zu gewähren, wurden Repository-Interfaces erstellt. Durch die Interfaces wurde die Persistenzschicht von der Domäne entkoppelt und austauschbar gehalten.

Spring Data JPA ermöglicht die Erstellung von Repository-Interfaces, die von `JpaRepository` erben. Dadurch werden automatisch CRUD-Methoden bereitgestellt, ohne dass eigener Code geschrieben werden muss. Für komplexere Abfragen wurden Custom Queries mit der `@Query`-Annotation und JPQL (Java Persistence Query Language) definiert.

Beispiel: Das `BookingRepository` enthält eine Custom Query zur Erkennung von Buchungskonflikten:

```
@Query("SELECT b FROM Booking b WHERE b.workspace.wid = :workspaceId
      AND b.date = :date " +
      "AND ((b.startTime <= :startTime AND b.endTime > :startTime) " +
      "OR (b.startTime < :endTime AND b.endTime >= :endTime) " +
      "OR (b.startTime >= :startTime AND b.endTime <= :endTime)))")
List<Booking> findConflictingBookings(
    @Param("workspaceId") Integer workspaceId,
    @Param("date") LocalDate date,
    @Param("startTime") LocalTime startTime,
```

```
@Param("endTime") LocalTime endTime);
```

Diese Query prüft auf zeitliche Überschneidungen zwischen einer neuen Buchung und bestehenden Buchungen für denselben Arbeitsplatz und dasselbe Datum.

4.3.2 Service-Klassen

Die Services implementieren die fachliche Logik und bilden die Use-Cases aus dem Anwendungsfall-diagramm ab. Jeder Service wurde als Spring-Bean mit der Annotation `@Service` gekennzeichnet.

Um zur Laufzeit eine Implementierung der Repository-Interfaces in den Services zu haben, wurde Constructor Injection eingesetzt. Der Konstruktor erhält die benötigten Repository-Ports als Parameter und wird von Spring automatisch mit den entsprechenden Implementierungen versorgt.

Gemäß der Hexagonalen Architektur arbeiten die Services nicht direkt mit Infrastructure-Repositories, sondern mit Port-Interfaces. Dies gewährleistet die Entkopplung der Geschäftslogik von der Infrastruktur:

```
@Service
public class BookingService implements BookingUseCase {

    private final BookingRepositoryPort bookingRepository;
    private final UserRepositoryPort userRepository;
    private final WorkspaceRepositoryPort workspaceRepository;
    private final UserPresenceUseCase userPresenceUseCase;

    public BookingService(BookingRepositoryPort bookingRepository,
                          UserRepositoryPort userRepository,
                          WorkspaceRepositoryPort workspaceRepository,
                          UserPresenceUseCase userPresenceUseCase) {

        this.bookingRepository = bookingRepository;
        this.userRepository = userRepository;
        this.workspaceRepository = workspaceRepository;
        this.userPresenceUseCase = userPresenceUseCase;
    }
}
```

Die Verwendung von Port-Interfaces (z.B. `BookingRepositoryPort` statt `BookingRepository`) ermöglicht es, die Persistenz-Implementierung auszutauschen, ohne den Service-Code zu ändern.

Implementierte Services: `BookingService`, `WorkspaceService`, `RoomService`, `UserService`, `UserPresenceService`, `UserRoomService` und `KeycloakProvisioningService`.

4.3.3 Data Transfer Objects (DTOs)

Für die Anwendungsfälle wurden passende DTOs erstellt, um die Entitäten für die REST-API zu transformieren. DTOs dienen der Entkopplung zwischen interner Domänenlogik und externer

Schnittstelle. Dadurch wird keine fachliche Logik nach außen gegeben und die über das Netzwerk zu übertragenden Daten werden auf das Notwendige reduziert.

Die DTOs wurden im Package `org.shared_desk.infrastructure.adapters.in.rest.*.dto` organisiert und nach ihrer Funktion benannt (z.B. `BookingRequestDto`, `BookingResponseDto`, `WorkspaceResponseDto`).

4.3.4 REST-Controller

Die REST-Controller bilden die Schnittstelle zwischen Frontend und Backend. Sie empfangen HTTP-Requests, delegieren die Verarbeitung an die entsprechenden Services und transformieren die Ergebnisse in HTTP-Responses.

Jeder Controller wurde mit `@RestController` und `@RequestMapping` annotiert. Die Endpunkte wurden mit `@GetMapping`, `@PostMapping`, `@PutMapping` und `@DeleteMapping` definiert.

Die Authentifizierung erfolgt über JWT-Tokens, die von Keycloak ausgestellt werden. Spring Security validiert automatisch die Tokens und extrahiert die Benutzerinformationen. Zur Autorisierung wurden Method-Level-Security-Annotations wie `@PreAuthorize("hasRole('ADMIN')")` eingesetzt.

4.4 Implementierung der Benutzeroberfläche

Die Benutzeroberflächen wurden mit Next.js 15 und React entwickelt. Next.js bietet Server-Side Rendering, einen App Router und optimierte Performance.

4.4.1 Projektstruktur und Komponenten

Die Frontend-Struktur folgt dem Next.js App Router Pattern. Alle Seiten befinden sich im `app/-` Verzeichnis, wiederverwendbare Komponenten im `components/-` Verzeichnis.

Es wurde ein allgemeines Layout erstellt (`layout.tsx`), das von allen Seiten wiederverwendet wird. Das Layout enthält die Navigation, das EDAG-Logo und den Benutzer-Avatar mit Logout-Funktion.

4.4.2 UI-Komponenten und Styling

Für die Gestaltung der Benutzeroberflächen wurde Tailwind CSS eingesetzt. Dabei wurden die EDAG-Farben verwendet (Grau: `#455561`, Rot: `#D71946`, Hellgrau: `#E4E5E6`).

Die UI-Komponenten basieren auf `shadcn/ui`, einer Sammlung wiederverwendbarer und zugänglicher React-Komponenten. Verwendet wurden unter anderem: `Button`, `Dialog`, `Card`, `Select`, `Calendar`, `Tooltip` und `Table`-Komponenten.

Die erstellten Mockups aus Abschnitt 3.2 bildeten die Grundlage für die Erstellung der Benutzeroberflächen. Aus ihnen konnten Position, Farbe und Größe der verschiedenen UI-Elemente abgeleitet werden.

4.4.3 Grafische Raumvisualisierung mit React Konva

Für die 2D-Darstellung der Räume mit Arbeitsplätzen wurde React Konva eingesetzt, eine React-Wrapper-Bibliothek für die Canvas-API. React Konva ermöglicht es, grafische Elemente deklarativ mit React-Komponenten zu erstellen.

Jeder Arbeitsplatz wird als Rechteck mit Rotation auf dem Canvas gezeichnet. Die Position wird aus den Datenbank-Koordinaten (`x_coord`, `y_coord`, `rotation`) berechnet. Der Verfügbarkeitsstatus wird durch Farbcodierung visualisiert:

- Grün: Verfügbar
- Rot: Gebucht
- Gelb: Fest zugewiesen
- Grau: Inaktiv

Die Arbeitsplätze sind interaktiv und reagieren auf Klicks. Bei einem Klick öffnet sich ein Dialog zur Buchung oder Detail-Ansicht. Für große Räume wurden Zoom- und Pan-Funktionen implementiert.

4.4.4 API-Integration und State Management

Die Kommunikation mit dem Backend erfolgt über Fetch-API-Calls. Für jeden Endpunkt wurde ein API-Client-Modul erstellt, das die HTTP-Requests kapselt und JWT-Tokens automatisch im Authorization-Header mitsendet.

Das State Management erfolgt mit React Hooks (`useState`, `useEffect`). Für häufig verwendete Logik wurden Custom Hooks erstellt, z.B. `useAuth()` für Authentifizierung, `useBookings()` für Buchungsverwaltung.

4.4.5 Internationalisierung

Mit next-intl wurde Mehrsprachigkeit für Deutsch und Englisch implementiert. Alle UI-Texte wurden in JSON-Dateien ausgelagert (`messages/de.json`, `messages/en.json`). Der Sprachwechsel erfolgt über einen Selector in der Navigation.

4.4.6 Fehlerbehandlung und Benutzerfreundlichkeit

Für eine ergonomische Gestaltung der Benutzeroberflächen wurde auf aussagekräftige Fehlermeldungen geachtet. Bei Validierungsfehlern werden die betroffenen Felder rot markiert und eine Fehlermeldung angezeigt.

Der Datenaustausch erfolgt asynchron, sodass die Benutzeroberfläche nicht einfriert. Loading-Spinner zeigen an, wenn Daten geladen werden. Erfolgsmeldungen werden als Toast-Notifications angezeigt.

Screenshots der fertigen Benutzeroberflächen befinden sich im Anhang (siehe Abschnitt ??). Es handelt sich um dieselben Ansichten, die auch in den Mockups aus Abschnitt 3.2 gezeigt wurden.

4.5 Testen der Anwendung

Wie in Abschnitt 3.6 (Maßnahmen zur Qualitätssicherung) beschrieben, wurde eine mehrstufige Teststrategie verfolgt.

4.5.1 Unit-Tests

Die Unit-Tests für die Services wurden mit JUnit 5 und Mockito geschrieben. Durch die Constructor Injection in den Service-Klassen war es möglich, mit Mockito Stubs der Repository-Interfaces zu erstellen und dem zu testenden Service zu übergeben.

Dadurch konnte das Datenbankverhalten vorgetäuscht werden und die Unit-Tests konnten isoliert von der Datenbank ausgeführt werden. Die Entkopplung hatte den Vorteil, dass die Laufzeit der Tests verringert und sie unabhängig von Fehlern auf Datenbankebene wurden.

Beispiel eines Unit-Tests:

```
@Test
void createBooking_shouldReturnBooking_whenValid() {
    // Arrange
    WorkspaceRepository mockRepo = Mockito.mock(WorkspaceRepository.class);
    BookingService service = new BookingService(mockRepo);
    // Act & Assert
    ...
}
```

Insgesamt wurden 41 Unit-Tests implementiert mit einer Code-Coverage von 72%.

4.5.2 API-Tests

Alle REST-Endpunkte wurden systematisch mit Swagger UI getestet. Dabei wurden sowohl Erfolgs- als auch Fehlerszenarien überprüft, inklusive der korrekten HTTP-Statuscodes (200, 201, 400, 401, 403, 404, 409) und Fehlermeldungen.

Die interaktive Swagger-Dokumentation unter `/swagger-ui.html` ermöglicht das direkte Testen aller CRUD-Operationen sowie komplexer Workflows wie Buchungsvorgänge mit Konflikterkennung.

4.5.3 Frontend-Tests

Die Benutzeroberfläche wurde durch manuelle Funktionstests validiert. Dabei wurde die Browser-Kompatibilität mit Chrome, Firefox, Edge und Safari sichergestellt. Besondere Aufmerksamkeit lag auf der Interaktivität der React-Konva-basierten Raumvisualisierung und der korrekten Darstellung in verschiedenen Bildschirmauflösungen.

API-Tests: Swagger-UI für alle Endpunkte (200, 201, 400, 401, 403, 404, 409).

Frontend-Tests: Manuelle Funktionstests, Browser-Kompatibilität (Chrome, Firefox, Edge, Safari).

Usability-Tests: 2 Testpersonen, 6 Aufgaben, Bewertung: 4.3/5.

Alle kritischen Fehler behoben. Code-Review ohne kritische Mängel.

4.6 Dokumentation

Entwickler: README (Backend/Frontend), Swagger-API-Doku (`/swagger-ui.html`), JavaDoc/JS-Doc, ER-Diagramme (Anhang ??), Docker-Doku, Styling-Guide.

Benutzer: Handbuch (Einführung, Buchung, Verwaltung, Admin, FAQ) mit Screenshots. Inline-Hilfe (Tooltips, Validierung).

4.7 Deployment und Abnahme

Deployment: Lokales MVP mit `docker-compose up -d` (60-90 Sek.). Container-Reihenfolge: PostgreSQL → Keycloak → Backend → Frontend.

Abnahme: 27.11.2025 durch Projektbetreuer - erfolgreich ohne Beanstandungen. Feedback: Saubere Architektur, umfassende Doku, benutzerfreundlich.

Pilotphase: 5 Testbenutzer, 45 Buchungen, keine kritischen Fehler, Bewertung positiv.

Produktiv-Einführung (geplant Q2 2026): Zentrale Server-Infrastruktur, HTTPS, Monitoring.

4.8 Retrospektive

Erfolgsfaktoren: Projekt in 80h abgeschlossen, gute Technologie-Wahl (Spring Boot/Next.js), saubere Architektur, Docker vereinfachte Deployment, gute Betreuer-Unterstützung.

Herausforderungen: Keycloak-Integration und React Konva zeitaufwendig.

Lessons Learned: Frühe Auseinandersetzung mit komplexen Integrationen, Docker von Anfang an, regelmäßige Reviews, frühes User-Feedback wertvoll.

4.9 Soll-/Ist-Vergleich

Phase	Soll (h)	Ist (h)	Differenz
Anforderungsspezifikation	8	8	0
Projektplanung	12	14	-1
Projektdurchführung	44	42	+1
Projektabschluss	16	16	0
Gesamt	80	80	0

Tabelle 4.1: Soll-Ist-Vergleich der Zeitplanung

4.10 Ausblick

Das entwickelte MVP bildet eine solide Grundlage für den produktiven Einsatz und zukünftige Erweiterungen:

- **Kurzfristig:** Deployment auf EDAG-Servern mit Hypertext Transfer Protocol Secure (HTTPS)-Verschlüsselung, Integration in das SSO-System, Monitoring und automatisierte Backups sowie Performance-Optimierung für größere Benutzerzahlen

- **Mittelfristig:** Analytics-Dashboard mit Auslastungsstatistiken und Reporting, Serienbuchungen, E-Mail-Benachrichtigungen, Outlook/Calendar-Integration sowie erweiterte Such- und Filterfunktionen
- **Langfristig:** KI-basierte Arbeitsplatzempfehlungen und Predictive Analytics, Multi-Ressourcen-Buchung (Besprechungsräume, Parkplätze), Mobile Apps für iOS/Android, Multi-Mandanten-Fähigkeit für standortübergreifende Nutzung

4.11 Fazit

4.11.1 Zielerreichung

Alle Projektanforderungen erfüllt: MVP in 80h, benutzerfreundliche UI, sichere Keycloak-Auth, alle Kernfunktionen, grafische Raumdarstellung, RBAC, vollständige Dokumentation.

4.11.2 Technische Bewertung

Stärken: Saubere Architektur (Hexagonale Architektur), moderne Technologien, klare Schichttrennung, Skalierbarkeit, hohe Wartbarkeit und Erweiterbarkeit, vollständige API-Dokumentation (Swagger).

Verbesserungspotenzial:

- **Bildspeicherung:** Profilbilder werden aktuell als BYTEA in der Datenbank gespeichert. Eine Auslagerung in einen Object Storage (z.B. MinIO, AWS S3) würde die Datenbankgröße reduzieren und die Performance verbessern.
- **Caching:** Implementierung eines Cache-Layers (z.B. Redis) für häufig abgerufene Daten wie Raum- und Arbeitsplatzinformationen würde die Response-Zeiten weiter reduzieren.
- **Automatisierte Tests:** Erweiterung der Test-Coverage durch End-to-End-Tests (z.B. mit Playwright) würde die Qualitätssicherung bei UI-Änderungen verbessern.
- **Monitoring:** Integration von produktionsreifen Monitoring-Tools (z.B. Prometheus, Grafana) für Systemmetriken und Application Performance Monitoring (APM).

4.11.3 Schlusswort

Die Entwicklung des Shared Desk Booking Systems war ein lehrreiches und erfolgreiches Projekt. Es demonstriert die praktische Anwendung moderner Softwareentwicklungs-Prinzipien und -Technologien im betrieblichen Kontext.

Die Anwendung bildet eine solide Grundlage für den produktiven Einsatz und kann schrittweise erweitert werden. Das positive Feedback der Testnutzer bestätigt, dass ein echter Mehrwert für die Mitarbeitenden geschaffen wurde.

Als Fachinformatiker für Anwendungsentwicklung konnte ich durch dieses Projekt meine Fähigkeiten in Full-Stack-Entwicklung, Architektur-Design und Projektmanagement unter Beweis stellen und weiter ausbauen.

A Anhänge

A.1 Datenbankentwurf (ER-Diagramme)

A.2 UML-Diagramme

A.2.1 Use-Case-Diagramme

Use-Case-Diagramm: Benutzer

Zeigt die grundlegenden Funktionen für alle angemeldeten Benutzer (7 Use Cases).

Datei: UseCase-Benutzer-Detail.puml

Use-Case-Diagramm: Raumadministrator

Zeigt die erweiterten Funktionen für Raumadministratoren (8 zusätzliche Use Cases).

Datei: UseCase-RoomAdmin-Detail.puml

Use-Case-Diagramm: Systemadministrator

Zeigt die globalen Verwaltungsfunktionen für Systemadministratoren (5 zusätzliche Use Cases).

Datei: UseCase-SystemAdmin-Detail.puml

Alle Diagramme befinden sich im Ordner: dokumentation/diagramme/

A.2.2 Aktivitätsdiagramme

A.2.3 Komponentendiagramme

Architektur-Komponentendiagramm

A.3 Screenshots der Benutzeroberfläche

Dieser Abschnitt zeigt Screenshots der wichtigsten Ansichten der Benutzeroberfläche. Die Benutzeroberfläche verwendet die EDAG-Farben und bietet eine intuitive, moderne Bedienung mit asynchronem Datenaustausch und aussagekräftigen Fehlermeldungen.

A.3.1 Login und Authentifizierung

Abbildung A.1: Keycloak Single Sign-On Login-Seite

A.3.2 Dashboard und Übersicht

Abbildung A.2: Dashboard mit Übersicht über nächste Buchungen. Oben rechts: Benutzermenü zum Abmelden oder Ändern des Profilbilds

Abbildung A.3: Raum-Übersicht mit Kachelansicht. Oben links: Sprachwechsel zwischen Deutsch und Englisch

A.3.3 Raum-Detailansicht und Arbeitsplatzbuchung

Abbildung A.4: Raum-Detailansicht mit grafischer 2D-Darstellung der Arbeitsplätze. Kalender und Tagesübersicht zeigen Homeoffice- und Office-Tage. Benutzer können ihren Status veröffentlichen und sehen, ob für den Tag bereits ein Arbeitsplatz reserviert ist

Abbildung A.5: Team-Anwesenheitsübersicht mit Filterung nach Datum und Büro

Abbildung A.6: Buchungsdialog mit Datums- und Zeitauswahl

A.3.4 Buchungsverwaltung

Abbildung A.7: Meine Buchungen - Übersicht aller eigenen Buchungen

Abbildung A.8: Buchung bearbeiten - Dialog

Abbildung A.9: Dialog zum Hinzufügen eines neuen Raums mit Eingabefeldern für Raumdetails

A.3.5 Anwesenheitsverwaltung

Abbildung A.10: Raumadministrator-Ansicht mit Möglichkeit zur Raumbearbeitung und Verwaltung zugewiesener Räume

Abbildung A.11: Hinzufügen neuer Arbeitsplätze im Raum. Dialog zur Definition von Arbeitsplatz-Eigenschaften

A.3.6 Administrationsbereich

Abbildung A.12: 2D-Editor für Arbeitsplätze. Benutzer können Arbeitsplätze per Drag-and-Drop verschieben und drehen

Abbildung A.13: Benutzerverwaltung für Raum. Zuweisung von Raumadministratoren und Berechtigungen

Abbildung A.14: Arbeitsplatzverwaltung innerhalb eines Raums. Übersicht aller Arbeitsplätze mit Bearbeitungsmöglichkeiten

A.3.7 System-Administration

Abbildung A.15: Buchungserstellung für Arbeitsplatz mit Auswahl von Datum und Zeitbereich

Abbildung A.16: Übersicht der Arbeitsplatzreservierungen mit Datum, Zeit und Status

Abbildung A.17: Teilweise Reservierung eines Arbeitsplatzes für einen bestimmten Zeitraum (z.B. 09:00-12:00 Uhr)

Abbildung A.18: Ganztagesreservierung eines Arbeitsplatzes für den gesamten Arbeitstag

Abbildung A.19: Kalenderfilterung zur Auswahl des gewünschten Reservierungsdatums